Statistical Review - 2026-07-23

Statistical Review — 2026-07-23

Status: resolved. Referenced by `engine/dd_wager.py`, `RISK_REGISTER.md` (risk #9), and `docs/white-paper.md`.

Why this review happened

Will reported that the Daily Double engine recommended $350 at 42% stated confidence and $4,200 (the full legal max) at 44% confidence, for identical scores. A two-point change in confidence should not swing a wager recommendation by 12x. He asked for the model to be reviewed "like a PhD thesis" and the mistakes fixed, not patched cosmetically.

Method

Read the full selection logic in `engine/dd_wager.py::analyze` as shipped before this review, reproduced the exact reported scenario (`scores=[4200, 6000, 3400]`, `p_correct=0.42` and `0.44`, `trials_per_wager=300`, `seed=42`), and checked whether the swing was a real feature of the decision or an artifact of how the "best" wager was chosen.

What was wrong

The old selection rule was:

```
best_wager = min(w for w, p in results if p >= best_prob - 1e-9)
```

Each `p` here is a Monte Carlo estimate of a true win probability, built from `trials_per_wager` simulated trials. Every such estimate carries a standard error of approximately `sqrt(p(1-p)/trials)` — at 300 trials and p near 0.5, that is roughly 2.9 percentage points, not a rounding-level amount. Comparing these estimates with a fixed `1e-9` tolerance treats them as if they had no error at all. When several candidate wagers have similar *true* win probabilities — a flat stretch of the underlying response surface, which is exactly what happens near $0 confidence swings like 42% vs. 44% — the `argmax` picks up whichever candidate happened to draw a slightly luckier set of random trials in that specific run. That is a textbook instance of **comparing noisy estimates as if they were exact**, and it is the direct cause of the reported 12x swing: nothing about the real decision changed that much between 42% and 44% confidence; the random seed did.

A second, independent defect was found in `engine/fj_bestresponse.py` (Tier 2 Final Jeopardy): the wager search treated a wager that only *ties* the leader's worst case identically to a wager that *beats it outright*, whenever both hit the same tie-inclusive win probability. At 100% self confidence there is no downside to betting one dollar more to convert a tie into a strict win, so recommending the tie was a real defect (a wager the model should never prefer once a strictly-dominant alternative is free), not a stylistic quirk.

The fixes

Daily Double (`engine/dd_wager.py`). Each candidate wager's standard error is now computed directly from its estimate (`_standard_error(p, trials) = sqrt(p(1-p)/trials)`). The wager reported as "best" is the *smallest* wager whose estimate is statistically indistinguishable from the single best estimate — the

gap between them is within `z_score` combined standard errors (`sqrt(se_w**2 + se_best**2)`), using a conservative default of `z_score=1.0` (roughly a one-standard-error band). Every wager meeting that bar is reported as the "plateau," so a wide plateau is shown honestly as "these are about equally good, the simulation can't tell them apart," rather than manufacturing false precision. `trials_per_wager`'s default was also raised from 800 to 3,000 to shrink the plateau in ordinary use.

Final Jeopardy Tier 2 (`engine/fj_bestresponse.py`). The exact-expectation enumeration now also tracks a *strict*-win probability (self strictly greater than every opponent's outcome, no tie) alongside the tie-inclusive one. Among wagers tied on tie-inclusive win probability, the recommendation now prefers whichever has the higher strict-win probability, then the smallest wager — so a free strict win is never passed over for a gratuitous tie.

Validation

- **Regression test**, `tests/test_dd_wager.py::test_small_confidence_change_no_longer_causes_wild_swings`: reproduces the exact reported scenario. Both 42% and 44% confidence now recommend $0, and each recommendation falls inside the *other* run's plateau — i.e., neither is a wager the other run would call statistically distinguishable from its own best.
- **Formula check**, `tests/test_dd_wager.py::test_standard_error_matches_binomial_formula`: confirms the standard error attached to each candidate matches `sqrt(p(1-p)/trials)` exactly.
- **Hand-derived test**, `tests/test_fj_bestresponse.py::test_analyze_prefers_strict_win_over_gratuitous_ti` same opponent model as the original hand-derivation (self=9000, opponent=5000), confirms wager 1001 (strict win, 87.5% win probability) is now preferred over wager 1000 (tie, same 87.5% win probability).
- **JS/Python parity re-check**: after porting both fixes into `app/index.html`, extracted the `<script>` block and ran it under Node.js against all of the above hand-derived and regression values. All 8 checks matched the Python engine's output exactly (see `CHANGELOG.md` entry for this date).
- Full test suite: 30/30 passing after these changes.

What this does NOT prove

- This review fixed a *selection* bug (how the "best" wager is chosen from noisy estimates); it did not calibrate or validate the underlying opponent behavior model, which remains an explicitly labeled placeholder (see `docs/adr/0002-opponent-model-assumptions.md`, risk #5). A statistically sound selection rule applied to an uncalibrated model still produces an uncalibrated recommendation — it is just no longer *additionally* corrupted by argmax noise.
- The plateau band communicates simulation uncertainty, not model uncertainty. A tight plateau means "the simulation is confident about this," not "this is the objectively correct wager in real play."
- The JS/Python parity check is a manual, one-time verification against a fixed set of hand-derived values, not an automated, continuously-run test. It does not prove the two engines will stay in sync as either changes — see risk #8 in `RISK_REGISTER.md`.
- Raising `trials_per_wager` shrinks the plateau but does not eliminate the need for it — any Monte Carlo estimate has *some* standard error, and argmax-over-noisy-estimates is a general pitfall, not specific to this trial count.